

How to use Inheritance in Application Development

In this article, I am going to discuss **How to use Inheritance in Application Development** with an Example using C# Language. Please read our previous two articles before proceeding to this article. Already in our previous two articles, we have discussed what is inheritance, the different rules of inheritance, How to Implement **Inheritance in C#** Language, and the **Different Types of Inheritance** supported by C# with classes.

What is Inheritance in C#?

The process of creating a new class from an existing class such that the new class acquires all the properties and behaviors of the existing class is called inheritance. The properties (or behaviors) are transferred from which class is called the superclass or parent class or base class whereas the class which derives the properties or behaviors from the superclass is known as a subclass or child class or derived class. In simple words, inheritance means to take something that is already made (or available).

Inheritance is the concept that is used for code reusability and changeability purpose. Here changeability means overriding the existed functionality or feature of the object or adding more functionality to the object.

How to use Inheritance in Application Development?

Inheritance is something that comes into the picture, not in the middle of a project or middle of application development. This can also come in the middle of the project development but generally when we start application development, in the initial stages only we plan inheritance and implement it in our project.

What is an Entity?

In DBMS terminology what is an Entity? An Entity is something that is associated with a set of attributes. An Entity can be a living or non-living object. But anything that is associated with a set of attributes is called Entity.

Remember, when we are going to develop an application, our application mainly deals with these Entities. Suppose, you are developing an application for a Bank. So, the Entity associated with the bank is a customer. A customer is an Entity. You are developing an application for a school; the Student will be the Entity. Suppose, you are developing an application for a business, then Employee is an entity. So, every application that we develop is associated with a set of entities.

Can we call a phone an Entity? Yes, we can call a phone an entity. It's not a living object but it can be called an entity. It has some attributes such as Company that manufacture the phone, the model number, the price, the size, the weight, etc.

The student is a living Entity. All students have some attributes such as Id, Name, Address, Phone number, and the class in which the student studies.

The employee is a living Entity. Employees have some attributes such as Employee number, name, job, department, Salary, etc.

Application Development Process:

So, generally, when we are developing an application, the process will be as follows.

Step1: Identify the Entities that are associated with the application

Suppose, we are developing an application for a School. Then for this Student Application, who are the entities. The Student is an Entity. TeachingStaff is an Entity. NonTeachingStaff is another Entity. Like this, we can identify the entities. So, in our application, we have identified three entities.

Entities: Student, TeachingStaff, NonTeachingStaff

Step2: Identify the attributes of each and every entity.

Entity: Student

Student Attributes: Id, Name, Address, Phone, Class, Marks, Grade, Fees

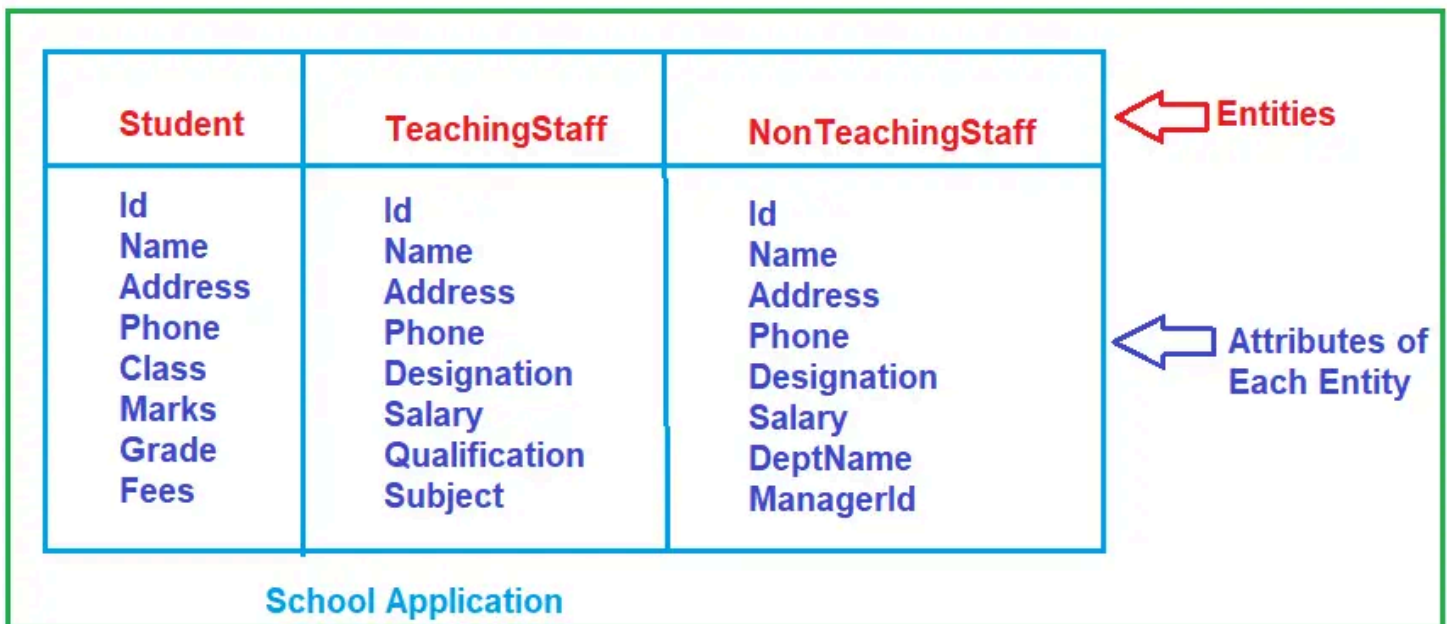
Entity: TeachingStaff

TeachingStaff Attributes: Id, Name, Address, Phone, Designation, Salary, Qualification, Subject

Entity: NonTeachingStaff

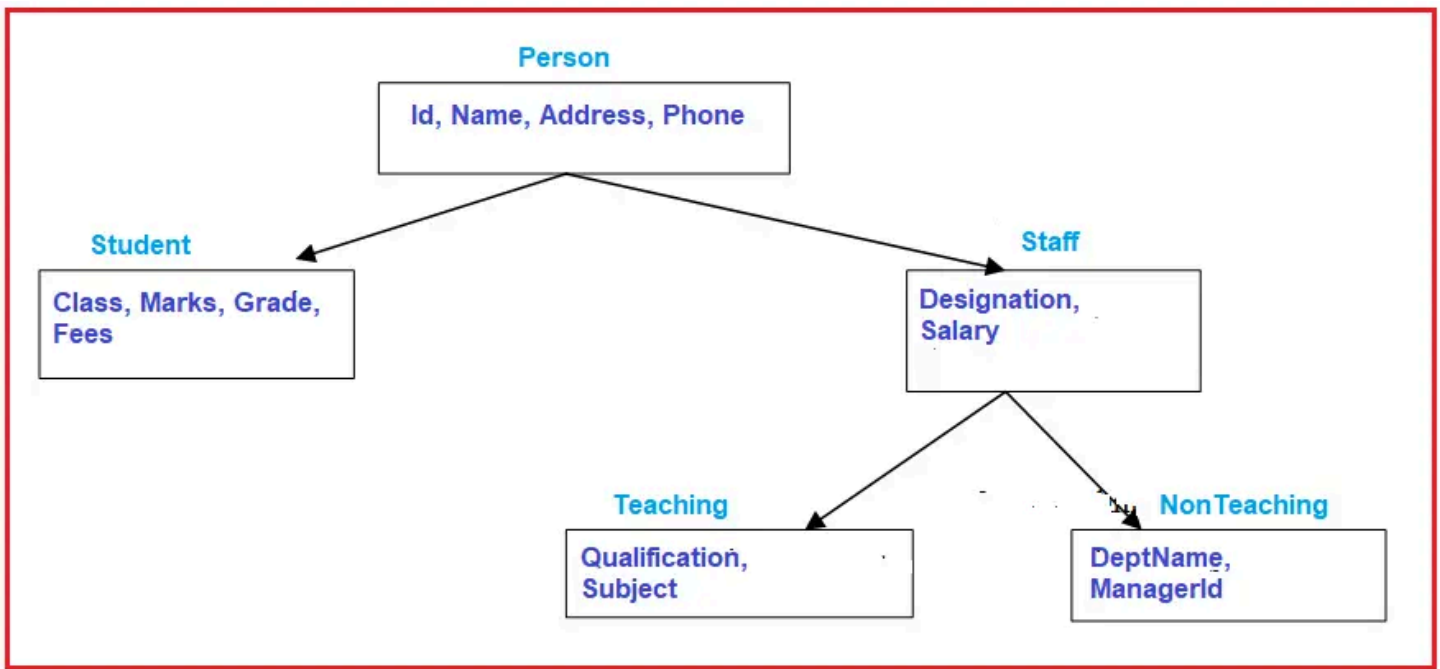
NonTeachingStaff Attributes: Id, Name, Address, Phone, Designation, Salary, DeptName, ManagerId

For a better understanding, please have a look at the below diagram.



Step3: Identify the common attributes and put them in a hierarchical order

See the following diagram for a better understanding. The Person contains four attributes Id, Name, Address, and Phone. Under the Person, we have Student and Staff. The Student inherits from the Person, so by default Student will get all those four attributes, and the rest of all other attributes are defined inside the Student Entity. Then we Staff Entity which is also inherited from Person and hence all those four common attributes are also available and plus we have defined the common attributes for Teaching and NonTeaching entities inside the Staff entity. So, Staff will contain six attributes. Finally, both Teaching and NonTeaching are inherited from the Staff Entity.



So, tomorrow if temporary staff comes into the picture, then also these properties are applicable to Temporary Staff. What you need to do is, create a new Entity with the specific properties and inherit it from the Staff entity.

Step4: Defining the classes that are representing the entities in Hierarchical order

After identifying of attributes of each entity, next, we need to define classes representing each and every entity. That is one class representing students, one class representing teaching staff, and another class representing the non-teaching staff. But, if we are defining three classes representing one entity, then there is a problem. The problem is there are some common attributes in each entity. So, if we start defining three classes individually, then there is code duplication. Why code duplication? See, we need to define Id three times, Name three times, Address three times, and Phone number three times. Like this, we have duplication in the code.

And we are working with an object-oriented programming language, not a procedure-oriented programming language and the strength of an object-oriented programming language is code re-usability, not code duplication. If we cannot make use of reusability, then there is no point coming into object-oriented programming.

So, what we need to do is we need to eliminate the redundancy or duplicates. See, in procedure-oriented language, we use structure to represent the entity. In DBMS we use tables to represent entities and in object-oriented programming, we use classes to represent entities.

Unfortunately, the structure does not support inheritance. A structure does not support inheritance, I require to define three entities with duplicate attributes. Id, Name, Address, and Phone number need to be duplicated in all three structures. And this is completely against the standard of object-oriented programming.

For all the three entities which are the common attributes? Id, Name, Address, and Phone are the common attributes. Let us put these common attributes in a class called Person. Once we define this class and once, we make this class a Parent class, then no need to define these attributes three times. One time we need to declare in the parent class and then we are consuming these properties under all the child classes. That means reusability comes into the picture.

```
public class Person
{
    public int Id;
    public string Name;
    public string Address;
    public string Phone;
}
```

Now we can define a class called Student inheriting from the Person class. And in the student class, we only need to define the Class, Marks, Grade, and Fees attributes as Id, Name, Address, and Phone are coming from the Person parent class.

```
public class Student : Person
{
    public int Class;
    public float Fees;
    public float Marks;
    public char Grade;
}
```

Next, you can create TeachingStaff and NonTeachingStaff classes inheriting from the Person class. But if you look at the TeachingStaff and NonTeachingStaff entities, apart from the four common attributes i.e. Id, Name, Address, Phone, these two entities also have another two common attributes i.e. Designation and Salary. Again, if we put these two properties in TeachingStaff and NonTeachingStaff classes, duplication comes. So, we need to create a separate class, let us call that class Staff and this Staff class inheriting from the Person class and in this class, we will put the two common properties i.e. Designation and Salary. So, now the Staff class has 6 attributes, four are coming from the Person class and two are defined in this class itself.

```
public class Staff : Person
{
    string Designation;
    double Salary;
}
```

Now, if we make the Staff class a parent class for TeachingStaff and NonTeachingStaff, then by default six attributes are coming. So, in the TeachingStaff we only need to define properties that are only for TeachingStaff such as Qualification and Subject. On the other hand, in the NonTeachingStaff, we only need to define the properties which are only for NonTeachingStaff such as DeptName and ManagerId. And both the TeachingStaff and NonTeachingStaff classes will inherit from the Staff class. Now, we are not going to call them TeachingStaff and NonTeachingStaff, rather we call them Teaching and NonTeaching as they are inheriting from the Staff.

```
public class Teaching : Staff
{
    string Qualification;
    string Subject;
}
public class NonTeaching : Staff
{
    string Deptname;
    string ManagerId;
}
```

So, this should be the process of how-to apply Inheritance in Application development.

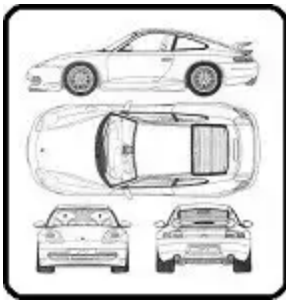
How to Make use of Inheritance in Realtime Application Development?

Generally, when we develop an application, we will be following a process as follows.

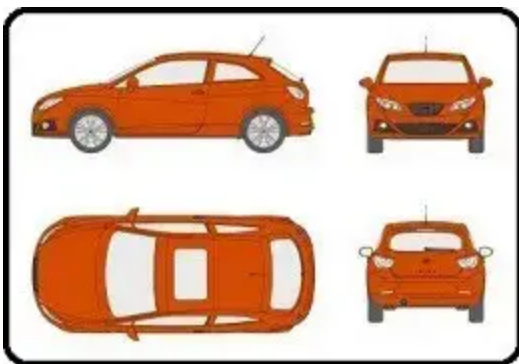
1. Identify the entity associated with the application
2. Identify the attribute that is associated with the application.
3. Now separate the attribute of each entity in a hierarchical order without having any duplicates.
4. Convert those entities into classes.

Another Realtime Example to Understand Inheritance in C#:

Acquiring the features of an existing class that is the base class into a new class that is derived class is known as inheritance. Inheritance is deriving a class from an existing class. So, what does it mean? Let us understand this through some real-time examples. We are taking the example of automobile engineering. Suppose, a car company has designed a car as shown in the below image.

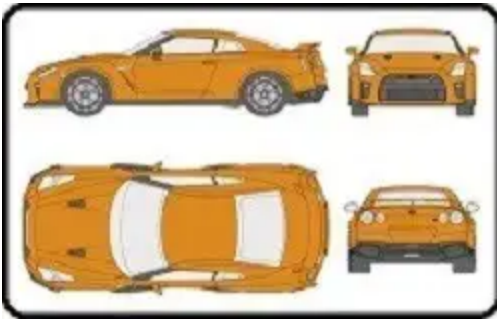


The above is just a design. Based on the above design they have manufactured a car as shown in the below image.



This is an actual car. So, automobile engineers will work on their design, and based on the design, the product is manufactured and many cars are manufactured and sold in the market. Now, this design is a successful one.

Now the company is trying to make a new model of the same car. So, do you think they have to do the design work from the beginning or again from the scratch? No, they can carry out the same features in a new design.



This is a new design. All the features have been borrowed from the previous design and extra features have been added. So, they can add multiple features they want to the old design. That means for making a new model which is matching with the first one, they don't have to do the redesigning work. Just they can extend the existing design and add new features to that design to come up with a new design. This is an example of inheritance. Acquiring the features of an existing design into a new design is nothing but inheritance.

Note: This is a task for you to design the classes for the above example and put it into the comment section.

In the next article, I am going to discuss [IS-A and HAS-A Relationship in C#](#) with Examples. Here, in this article, I try to explain **How to use inheritance in Application Development** with an Example using C# Language and I hope you enjoy this How to use Inheritances in Application Development with an Example using C# Language article.